



Introduction to Azure Service Bus



What is Azure Service Bus?

Azure Service Bus is a messaging service on cloud used to connect any applications, devices, and services running in the cloud to any other applications or services.

Service Bus Services

- Queue
- Topic

Service Bus Queue

Queues messaging service offers First In First Out data delivery.

It enables ordered handling of unbounded sequences of related messages.

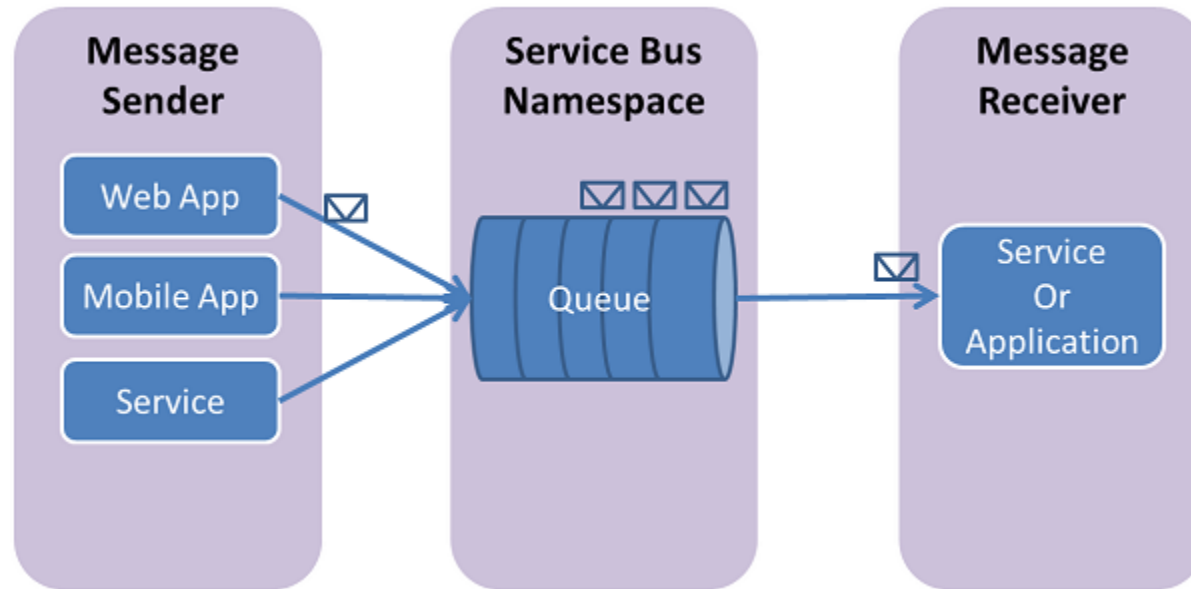
Message sessions is used to ensure a first-in, first-out (FIFO).

Queues enable us to store messages until the receiving application is available to receive and process them.

Messages in queues are timestamped on arrival and ordered according to it.

Messages are delivered on request (Pull mode delivery). Queue is often used for point-to-point communication.

Queue Bus Service



Service Bus Topic

Topics is nothing but single input queue but multiple output queues.

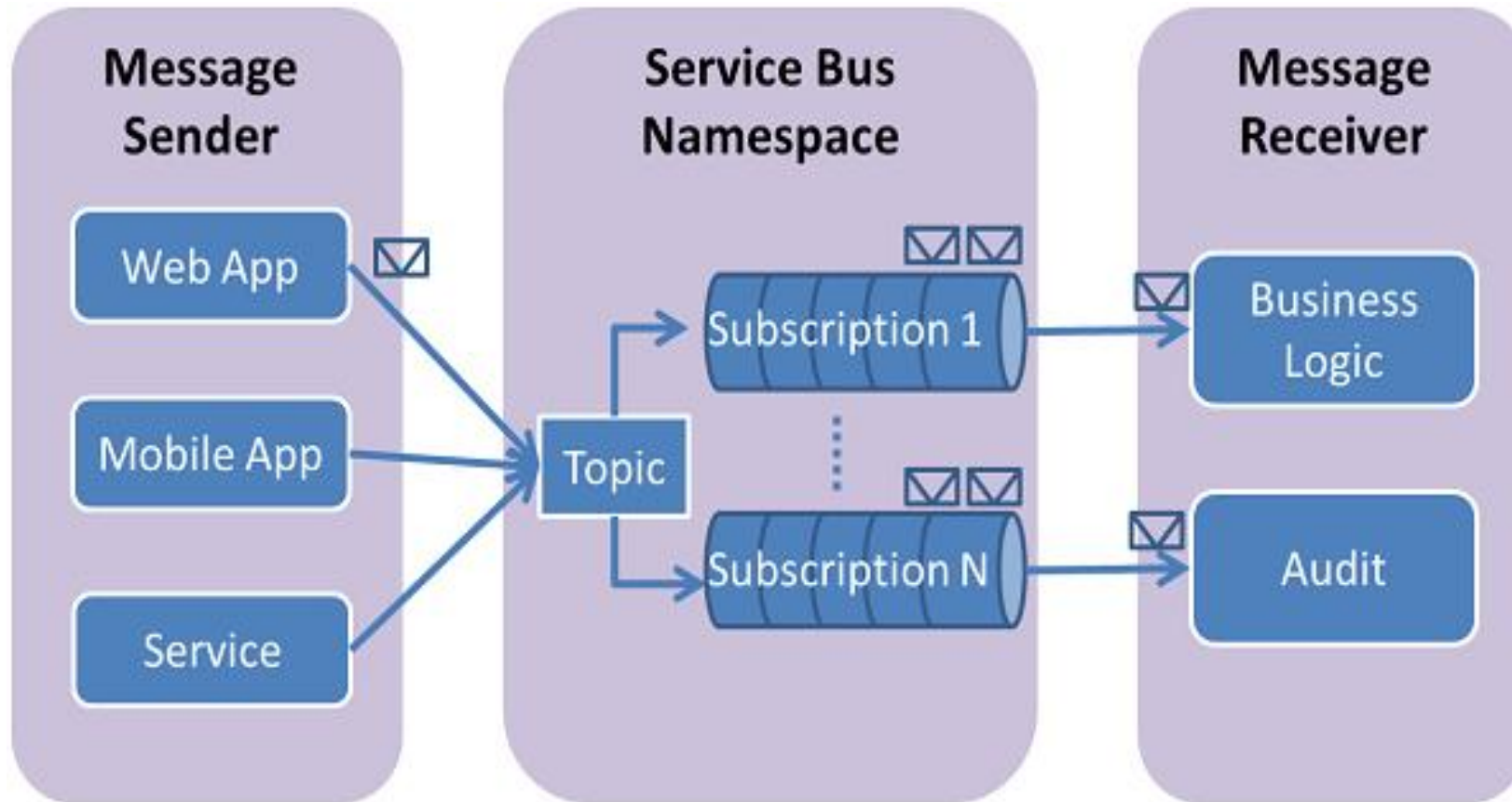
Output queues are nothing but Subscriptions.

Receiving apps needs to create subscriptions to receive messages from Topic.

Subscription can filter out the messages based on rules on message properties.

Publish/subscribe scenarios can be implemented using Topics.

Service Bus Topic



Service Bus Queues vs Topics

Similarities

Supports unidirectional communication.

Supports FIFO message delivery.

Differences

Queues can be used for one to one communication but Topics can be used for one to many communication scenario.

Receivers needs to subscribe for topics and optionally

Create Service Bus Namespace

A namespace is a scoping container for all messaging components. Multiple queues and topics can reside within a single namespace, and namespaces often serve as application containers.

Create Service Bus Namespace

To create a namespace:

1. Sign in to the Azure portal
2. In the left navigation pane of the portal, select + Create a resource, select Integration, and then select Service Bus.

Create Service Bus Namespace

The screenshot displays the Microsoft Azure portal interface. At the top, the header reads 'Microsoft Azure' with a search bar for 'Search resources, services, and docs (G+/I)'. The left-hand navigation pane includes links to 'Home', 'Dashboard', 'All services', and a 'FAVORITES' section with items like 'Resource groups', 'All resources', 'Recent', 'Virtual machines', 'Subscriptions', 'Help + support', 'Advisor', 'Event Hubs', 'Service Bus', 'Relays', 'DevTest Labs', 'Lab accounts', 'Azure Active Directory', 'Cost Management + Billing', 'Storage accounts', 'Monitor', and 'Azure compute galleries'. The 'Create a resource' button at the top of this pane is highlighted with a red box. The main content area is titled 'Create a resource' and features a search bar for 'Search services and marketplace'. Below this, a list of categories is shown, with 'Integration' highlighted by a red box. To the right of the categories, a list of 'Popular Azure services' is displayed, including 'API Management', 'Data Catalog', 'Data Factory', 'Logic App', 'Azure API for FHIR', 'Integration Account', 'Logic Apps Custom Connector', 'Integration Service Environment', and 'Service Bus'. The 'Service Bus' entry is highlighted with a red box and includes links to 'Create', 'Docs', and 'MS Learn'.

Create Service Bus Namespace

3. In the Basics tag of the Create namespace page, follow these steps:

- a. For Subscription, choose an Azure subscription in which to create the namespace.
- b. For Resource group, choose an existing resource group in which the namespace will live, or create a new one.
- c. Enter a name for the namespace. The namespace name should adhere to the following naming conventions:

The name must be unique across Azure. The system immediately checks to see if the name is available.

- The name length is at least 6 and at most 50 characters.
- The name can contain only letters, numbers, hyphens “-”.
- The name must start with a letter and end with a letter or number.
- The name doesn't end with “-sb” or “-mgmt”.

Create Service Bus Namespace

- d. For Location, choose the region in which your namespace should be hosted.
- e. For Pricing tier, select the pricing tier (Basic, Standard, or Premium) for the namespace. For this quickstart, select Standard.
- f. Select Review + create. The system now creates your namespace and enables it. You might have to wait several minutes as the system provisions resources for your account.

Create Service Bus Namespace

F. On the Create page, review settings, and select Create.

 **Create namespace** ...
Service Bus

Basics Networking Tags Review + create

Project Details
Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Visual Studio Enterprise Subscription

Resource group *

spsbusrg

[Create new](#)

Instance Details
Enter required settings for this namespace.

Namespace name *

contososbusns0426

.servicebus.windows.net

Location *

East US

Pricing tier *

Standard

[Browse the available plans and their features](#)

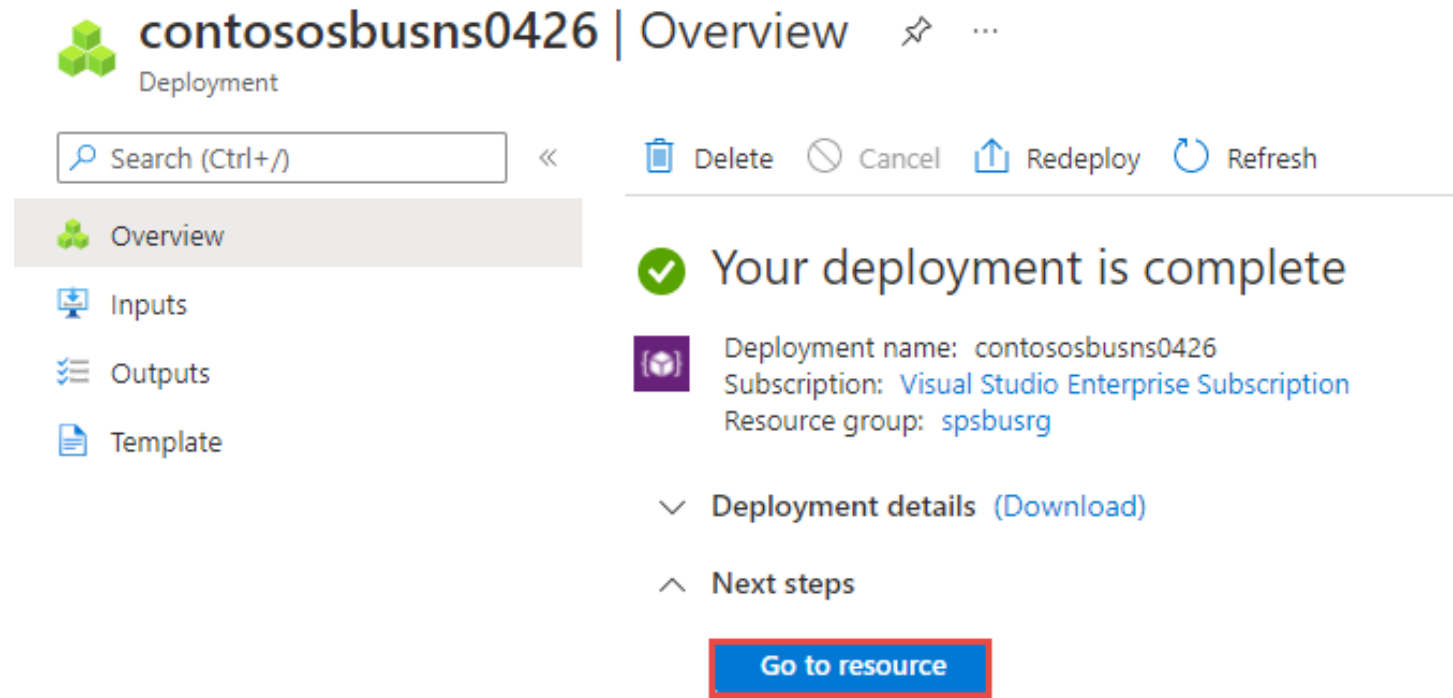
Review + create

< Previous

Next: Networking >

Create Service Bus Namespace

g. Select Go to resource on the deployment page.



The screenshot shows the Azure portal interface for a deployment named 'contososbusns0426'. The page title is 'contososbusns0426 | Overview'. Below the title, there is a search bar and a navigation pane on the left with links to 'Overview', 'Inputs', 'Outputs', and 'Template'. The 'Overview' link is selected. On the right, there are action buttons: 'Delete', 'Cancel', 'Redeploy', and 'Refresh'. A green checkmark icon indicates that the deployment is complete. Below this, the deployment details are listed: 'Deployment name: contososbusns0426', 'Subscription: Visual Studio Enterprise Subscription', and 'Resource group: spsbusrg'. There are expandable sections for 'Deployment details (Download)' and 'Next steps'. A red-bordered button labeled 'Go to resource' is highlighted at the bottom.

contososbusns0426 | Overview

Deployment

Search (Ctrl+ /)

Overview

Inputs

Outputs

Template

Delete Cancel Redeploy Refresh

✓ Your deployment is complete

Deployment name: contososbusns0426
Subscription: Visual Studio Enterprise Subscription
Resource group: spsbusrg

Deployment details (Download)

Next steps

Go to resource

Create a Queue in the Azure portal

1. On the Service Bus Namespace page, select Queues in the left navigational menu.
2. On the Queues page, select + Queue on the toolbar.
3. Enter a name for the queue, and leave the other values with their defaults.
4. Now, select Create.

Create a Queue in the Azure portal

Dashboard > contososbusns > contososbusns

contososbusns | Queues ...
Service Bus Namespace

Search (Ctrl+ /)

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems

Settings

Shared access policies
Geo-Recovery
Migrate to premium
Encryption
Properties
Locks

Entities

Queues
Topics

Monitoring

Insights (Preview)
Alerts
Metrics

+ Queue 2 refresh

Search to filter items...

Name	Status
No results.	

Create queue

Service Bus

Name * ①
myqueue 3

Max queue size
1 GB

Max delivery count * ①
10

Message time to live ①
Days: 14, Hours: 0, Minutes: 0, Seconds: 0

Lock duration ①
Days: 0, Hours: 0, Minutes: 0, Seconds: 30

☐ Enable auto-delete on idle queue ①
☐ Enable duplicate detection ①
☐ Enable dead lettering on message expiration ①
☐ Enable partitioning ①
☐ Enable sessions ①
☐ Forward messages to queue/topic ①

Create 4

Create a Topic in the Azure portal

1. On the Service Bus Namespace page, select Topics on the left menu.
2. Select + Topic on the toolbar.
3. Enter a name for the topic. Leave the other options with their default values.
4. Select Create.

Create a Topic in the Azure portal

Dashboard > contososbusns0426 > contososbusns0426

contososbusns0426 | Topics ...
Service Bus Namespace

Search (Ctrl+ /) << **+ Topic** 2 Refresh

Search to filter items...

Name	Status	Max size
No results.		

Entities

- Queues
- Topics** 1

Monitoring

- Insights (Preview)
- Alerts
- Metrics
- Diagnostic settings
- Logs
- Workbooks

Create topic Service Bus

Name * ⓘ **mytopic** 3 ✓

Max topic size ⓘ
1 GB

Message time to live ⓘ

Days	Hours	Minutes	Seconds
14	0	0	0

☐ Enable auto-delete on idle topic ⓘ

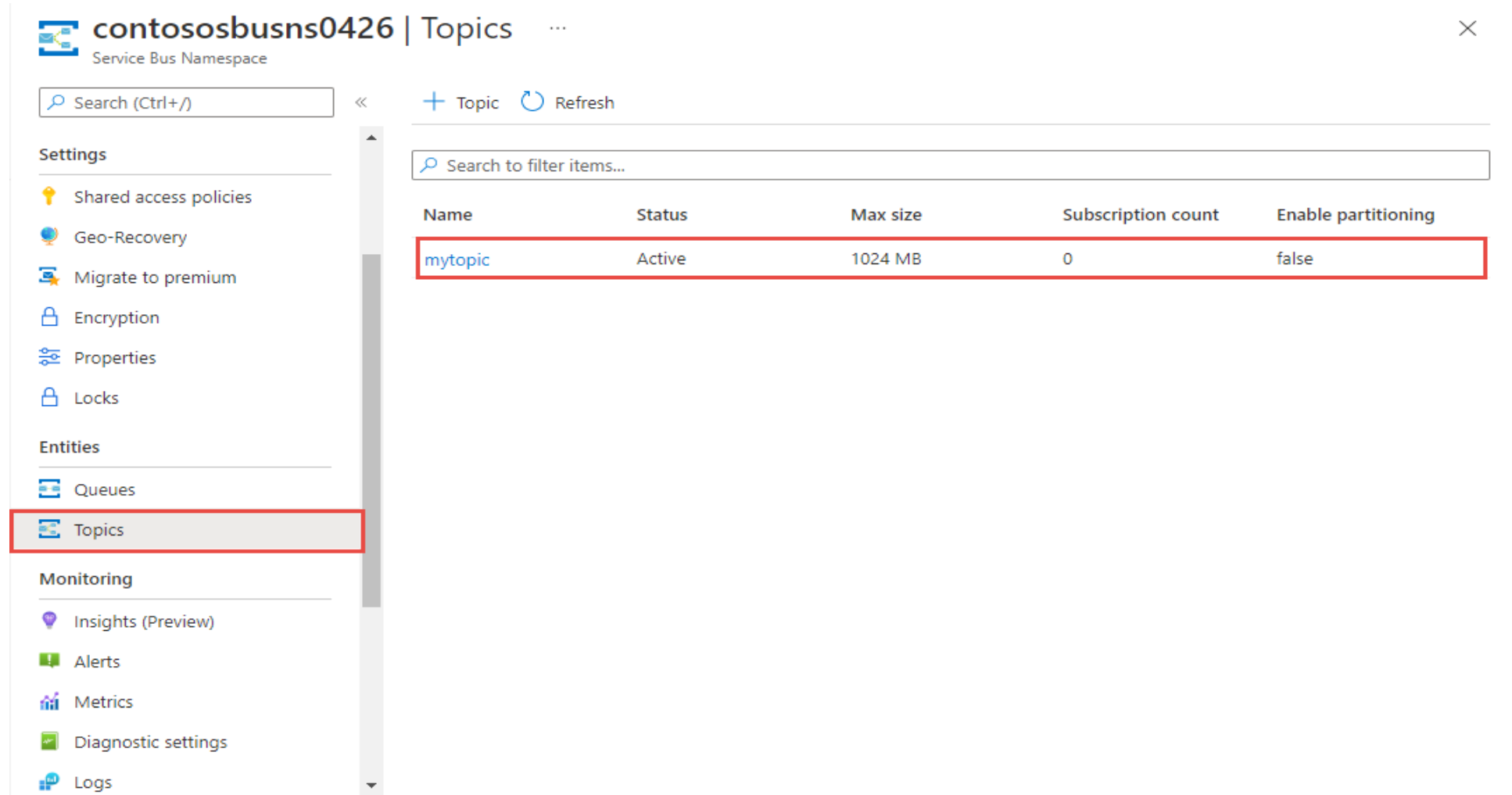
☐ Enable duplicate detection ⓘ

☐ Enable partitioning ⓘ

Create 4

Create subscriptions to the topic

1. Select the topic that you created in the previous section.

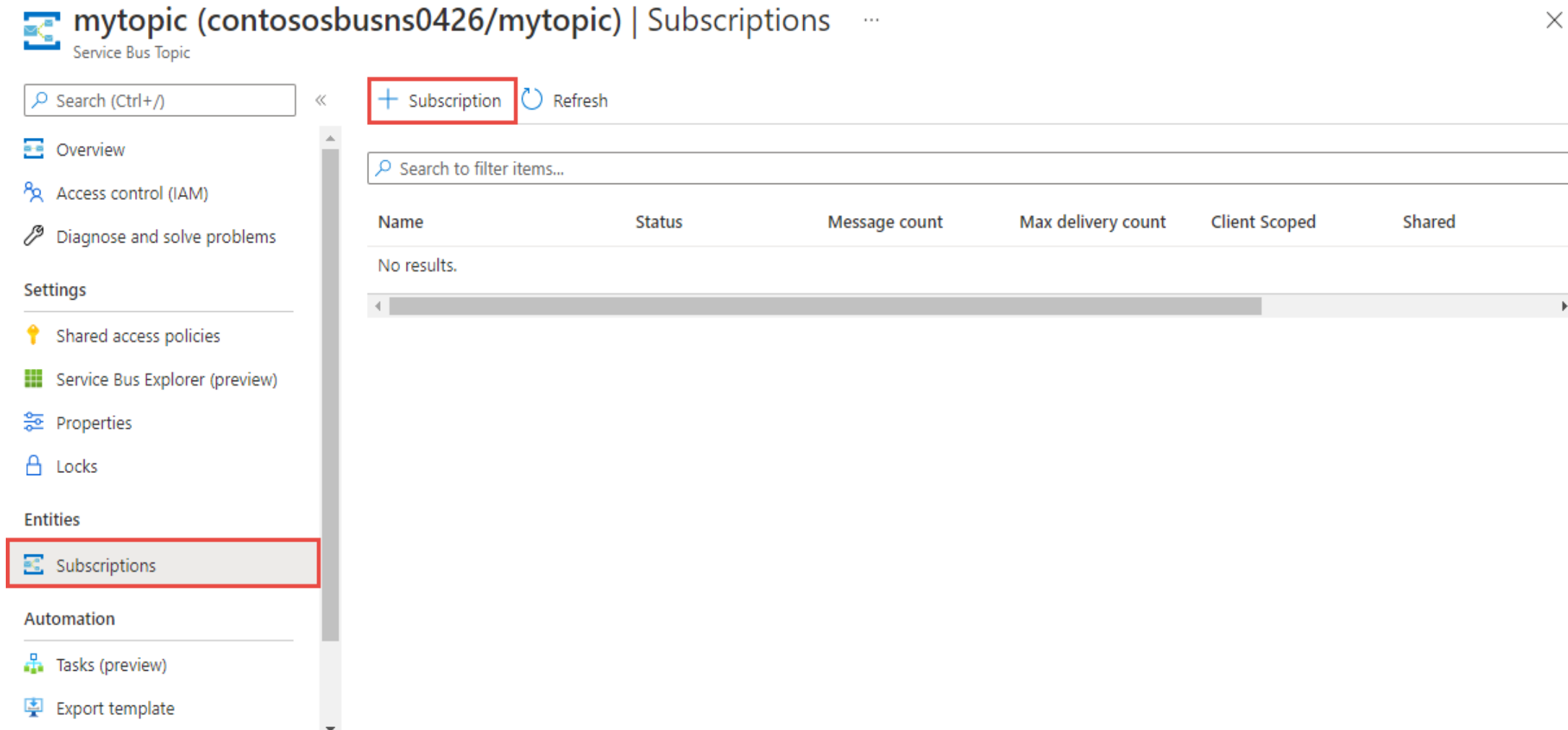


The screenshot shows the Azure Service Bus interface for the namespace **contososbusns0426**. The left-hand navigation pane is expanded, and the **Topics** option is selected and highlighted with a red border. The main content area displays a table of topics. A search bar at the top of the table contains the text "Search to filter items...". The table has five columns: **Name**, **Status**, **Max size**, **Subscription count**, and **Enable partitioning**. A single topic, **mytopic**, is listed and highlighted with a red border. The table data is as follows:

Name	Status	Max size	Subscription count	Enable partitioning
mytopic	Active	1024 MB	0	false

Create subscriptions to the topic

2. On the Service Bus Topic page, select Subscriptions from the left menu, and then select + Subscription on the toolbar.



The screenshot shows the Azure portal interface for a Service Bus Topic named 'mytopic' (contosobusns0426/mytopic). The left-hand navigation pane is visible, with the 'Subscriptions' option highlighted with a red rectangle. The main content area displays the 'Subscriptions' page, which includes a search bar, a toolbar with a '+ Subscription' button (highlighted with a red rectangle) and a 'Refresh' button, and a table with columns: Name, Status, Message count, Max delivery count, Client Scoped, and Shared. The table currently shows 'No results.' and a horizontal scrollbar is visible below it.

mytopic (contosobusns0426/mytopic) | Subscriptions

Service Bus Topic

Search (Ctrl+ /)

Overview

Access control (IAM)

Diagnose and solve problems

Settings

Shared access policies

Service Bus Explorer (preview)

Properties

Locks

Entities

Subscriptions

Automation

Tasks (preview)

Export template

+ Subscription Refresh

Search to filter items...

Name	Status	Message count	Max delivery count	Client Scoped	Shared
No results.					

Create subscriptions to the topic

3. On the Create subscription page, follow these steps:

1. Enter name of the subscription (example S1)
2. Enter Max delivery count (example 3)
3. Then, select Create to create the subscription.

Create subscriptions to the topic

Create subscription ...

Service Bus

Name * ⓘ

S1 ✓

Max delivery count * ⓘ

3 ✓

Auto-delete after idle for ⓘ

Days Hours Minutes Seconds

14 0 0 0

☐ Never auto-delete

☐ Forward messages to queue/topic ⓘ

MESSAGE SESSIONS

Service bus sessions allow ordered handling of unbounded sequences of related messages. With sessions enabled a subscription can guarantee first-in-first-out delivery of messages. [Learn more.](#)

☐ Enable sessions

MESSAGE TIME TO LIVE AND DEAD-LETTERING

Message time to live (default) ⓘ

Days Hours Minutes Seconds

14 0 0 0

☐ Enable dead lettering on message expiration

☐ Move messages that cause filter evaluation exceptions to the dead-letter subqueue

MESSAGE LOCK DURATION

Lock duration ⓘ

Days Hours Minutes Seconds

0 0 0 30

Create

4. Repeat the previous step twice to create subscriptions named S2 and S3.



AZURE SERVICE BUS

MODULE: APPLICATION DEVELOPMENT 3B

CONTENT DEVELOPED BY: CASSIM VANKER

LEARNING OBJECTIVES

- Azure Service Bus
 - What is Azure Service Bus?
 - Fundamental's of Azure Service Bus
 - Azure Service Bus Queues
 - Azure Service Bus queues – ReceiveAndDelete
 - Azure Service Bus queues – Peeklock
 - Azure Service Bus queues – Senarios
 - Azure Service Bus Topics
 - Azure Service Bus Relays

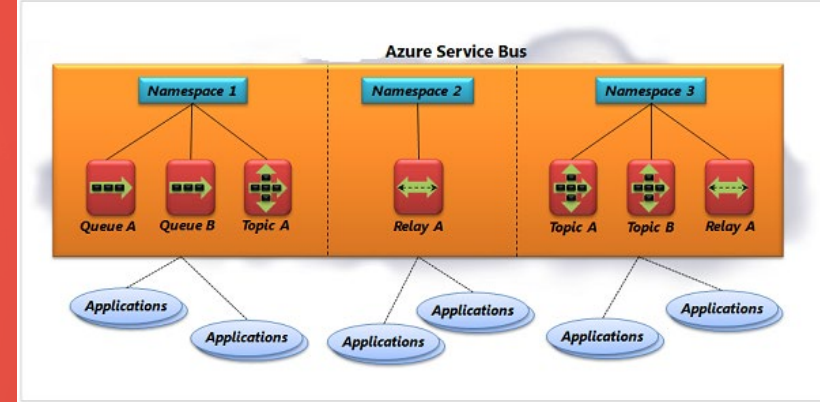
WHAT IS AZURE SERVICE BUS?

- Whether an application or service runs in the cloud or on premises, it often needs to interact with other applications or services.
- To provide a broadly useful way to do this, Microsoft Azure offers Service Bus.

FUNDAMENTALS OF AZURE SERVICE BUS

- Different application requirements call for different styles of communication.
- Sometimes, letting applications send and receive messages through a simple queue is the best solution.
- In other situations, an ordinary queue isn't enough; a queue with a publish-and-subscribe mechanism is better.
- However in some cases, all that's needed is a connection between applications, and queues are not required.
- Azure Service Bus provides all three options, enabling your applications to interact in several different ways.

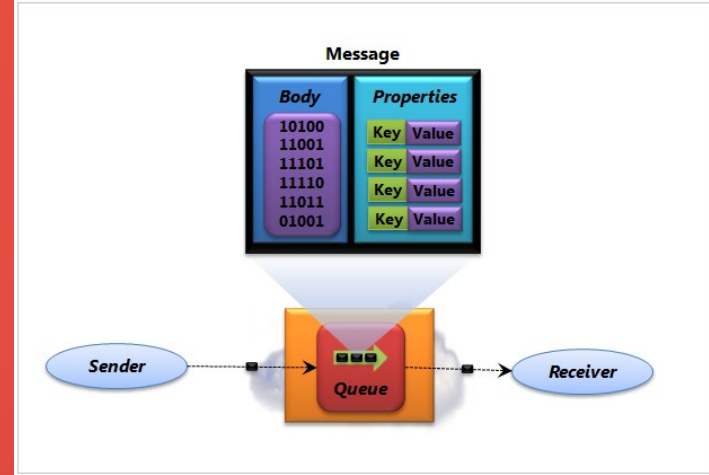
FUNDAMENTALS OF AZURE SERVICE BUS



Three different communication mechanisms, are supported using Azure Service Bus:

- **Queues:** allow one-directional communication. Each queue acts as an intermediary (sometimes called a *broker*) that stores sent messages until they are received. Each message is received by a single recipient.
- **Topics:** one-directional communication using **subscriptions**. A single topic can have multiple subscriptions. Like a queue, a topic acts as a broker, but each subscription can optionally use a filter to receive only messages that match specific criteria.
- **Relays:** bi-directional communication. Unlike queues and topics, a relay doesn't store messages; it's not a broker. Instead, it just passes them on to the destination application.
- When you create a queue, topic, or relay, you give it a name. Combined with whatever you called your namespace, this name creates a unique identifier for the object.

AZURE SERVICE BUS QUEUES



- A sender sends a message to a Service Bus queue, and a receiver consumes that message at some later time.
- A queue can have just a single receiver. Or, multiple applications can read from the same queue.
- Each message has two parts: a set of properties, a key/value pair, and a message payload
- The payload can be binary, text, or even XML.

AZURE SERVICE BUS QUEUES

- A receiver can read a message from a Service Bus queue differently.
- The first option, called **ReceiveAndDelete**.
- This option Receives a message from the queue and deletes it after event handling is completed.
- However, if the receiver crashes before it finishes processing the message, the message is lost.

AZURE SERVICE BUS QUEUES - RECEIVEANDDELETE

- A receiver can read a message from a Service Bus queue in two different ways.
- The first option, called **ReceiveAndDelete**.
- This option Receives a message from the queue and immediately deletes it. This option is simple
- However, if the receiver crashes before it finishes processing the message, the message is lost. (As it's been removed from the queue).

AZURE SERVICE BUS QUEUES - PEEKLOCK

- Default way to receive messages
- Like **ReceiveAndDelete**, a **PeekLock** read removes a message from the queue. It doesn't delete the message. Instead, it locks the message, making it invisible to other receivers, and then waits for one of three events:
 - If the receiver processes the message successfully, it calls `CompleteMessageAsync()`, and the queue deletes it.
 - If the receiver decides it can't process the message successfully, it calls `AbandonMessageAsync()`. The queue then removes the lock from the message and makes it available to other receivers.
 - If the receiver calls neither of these methods within a configurable period of time (by default, 60 seconds), the queue assumes the receiver has failed. In this case, it behaves like the receiver called `Abandon`, making the message available to other receivers.

AZURE SERVICE BUS QUEUES - PEEKLOCK

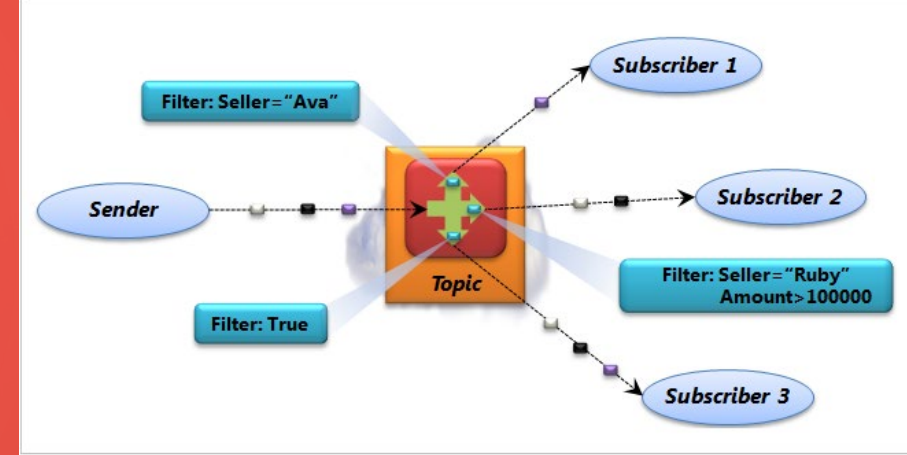
- Using this pattern a message could be delivered twice.
- As such, applications using Service Bus queues must be prepared for this event.
- To make duplicate detection easier, each message has a unique *MessageID* property that by default stays the same no matter how many times the message is read from a queue.

AZURE SERVICE BUS QUEUES - SENARIOS

- They enable applications to communicate even if both apps are not running at the same time.
- Something that's especially handy with batch and mobile applications.
- A queue with multiple receivers also provides automatic load balancing, since sent messages are spread across these receivers.

AZURE SERVICE BUS TOPICS

- A *topic* is similar in many ways to a queue.
- Senders submit messages to a topic in the same way that they submit messages to a queue, and those messages look the same as with queues.
- The difference is that topics enable each receiving application to create its own *subscription* and optionally define a *filter*.
- A subscriber receives a copy of each message in the topic, but by using a filter, can then receive only the messages that match that filter.
- As with queues, subscribers to a topic can read messages using either `ReceiveAndDelete` or `PeekLock`.
- Unlike queues, however, a single message sent to a topic can be received by multiple subscriptions.
- This approach, commonly called publish and subscribe (or pub/sub), is useful whenever multiple applications are interested in the same messages.



Introduction to Service Bus Demo Code

- Overview of Service Bus in Azure
- Introduction to the `ServiceBusDemo` namespace
- Purpose of the `ServiceBusQueuesManager` class

Key Dependencies

- ``using Azure.Messaging.ServiceBus;``
- Role of Service Bus in messaging
- Importance of the `ServiceBusClient`, `ServiceBusProcessor`, and `ServiceBusSender` classes

Class Overview

- Static class designed to manage Service Bus queues
- Three main components:
 - - `ServiceBusClient` (_client)
 - - `ServiceBusProcessor` (_processor)
 - - `ServiceBusSender` (_sender)

Initialization Method

- ``InitializeClient`` method
- Checks if the client is null, then initializes:
- - ServiceBusClient with options
(``ServiceBusTransportType.AmqpWebSockets``)
- The lock statement prevents multiple threads from entering the critical section of code simultaneously.
- - ServiceBusProcessor with options
(``ServiceBusReceiveMode.PeekLock``,
``MaxConcurrentCalls = 1``)

Initialization Method

- Setting AutoCompleteMessages to True: ServiceBusProcessor automatically calls CompleteMessageAsync on your behalf after the ProcessMessageAsync handler finishes executing without throwing an exception
- ReceiveMode is a property in Azure Service Bus that determines how messages are received and processed from a queue or subscription:
 - ReceiveMode.Queue: Messages are received from the queue and are automatically deleted from the queue after they are processed.
 - ReceiveMode.Subscription: Messages are received from the subscription and are automatically deleted from the subscription after they are processed.

Initialization Method

- $\text{Set}(\text{ur}, \text{th}, \text{ne}, \text{ss}, \text{ag}, \text{e}, \text{rs}, \text{oc}, \text{es}, \text{ss}, \text{os}, \text{and}, \text{ne}, \text{ss}, \text{ag}, \text{e}, \text{sen}, \text{des})$

$\text{rs}, \text{oc}, \text{es}, \text{ss}, \text{os}$

$\text{cl}, \text{ien}, \text{t}, \text{cs}, \text{eat}, \text{fe}, \text{rs}, \text{oc}, \text{es}, \text{ss}, \text{os}$ $\text{arr}, \text{de}, \text{w}, \text{ru}, \text{eu}, \text{e}$

$\text{rs}, \text{oc}, \text{es}, \text{ss}, \text{os}$ $\text{ort}, \text{ion}, \text{ss}$

sen, des $\text{cl}, \text{ien}, \text{t}, \text{cs}, \text{eat}, \text{fe}, \text{sen}, \text{des}$ $\text{arr}, \text{de}, \text{w}, \text{ru}, \text{eu}, \text{e}$

- $\text{E}, \text{wen}, \text{t}, \text{ss}, \text{cs}, \text{ir}, \text{t}, \text{ion}, \text{ss}$

- $\text{Set}(\text{ur}, \text{th}, \text{ew}, \text{en}, \text{t}, \text{th}, \text{at}, \text{x}, \text{il}, \text{h}, \text{and}, \text{le}, \text{th}, \text{ne}, \text{ss}, \text{ag}, \text{e}, \text{rs}, \text{oc}, \text{es}, \text{ss}, \text{ing}, \text{and}, \text{ess}, \text{os}, \text{h}, \text{and}, \text{ling})$

$\text{rs}, \text{oc}, \text{es}, \text{ss}, \text{os}$ $\text{rs}, \text{oc}, \text{es}, \text{ss}, \text{ne}, \text{ss}, \text{ag}, \text{e}, \text{as}, \text{yn}, \text{c}$ $\text{ne}, \text{ss}, \text{ag}, \text{e}, \text{h}, \text{and}, \text{le}, \text{ss}$

$\text{rs}, \text{oc}, \text{es}, \text{ss}, \text{os}$ $\text{rs}, \text{oc}, \text{es}, \text{ss}, \text{ess}, \text{os}, \text{as}, \text{yn}, \text{c}$ $\text{ess}, \text{os}, \text{h}, \text{and}, \text{le}, \text{ss}$

Receiving Messages

- `StartReceivingMessagesAsync` method:
 - - Calls `InitializeClient`
 - - Starts processing messages
- `StopReceivingMessagesAsync` method:
 - - Stops message processing

Sending Message Batch

- `SendMessageBatchAsync` method:
 - - Initializes the client
 - - Creates a batch of messages
 - - Sends the batch to the queue
 - - Handles errors during message sending

Message & Error Handlers

- `MessageHandler` method:
 - - Processes each message received
 - - Completes the message to remove it from the queue
- `ErrorHandler` method:
 - - Logs any errors encountered during processing

Cleanup Resources

- `DisposeAsync` method:
 - - Disposes of `ServiceBusProcessor`,
`ServiceBusClient`, and `ServiceBusSender`
 - - Ensures resources are properly released

Summary

- Overview of how `ServiceBusQueuesManager` handles Azure Service Bus queues
- Importance of message processing, error handling, and resource management
- Recap of methods: initialization, message handling, and cleanup